
Tutorial 6 Solution

Securing Web Applications

Computer Security

BCS2420

Part A: Multiple Choice - Ideal Answers

1. **Answer:** D. SOP does not apply if two pages share the same top-level domain (TLD, e.g., `.com`).
Explanation: SOP checks scheme, host, port, not just TLD. So `evil.com` vs `bank.com` are different origins even though they share `.com`.
2. **Answer:** B. The browser automatically fetches malicious scripts from a compromised website, exploiting vulnerabilities without explicit user action.
3. **Answer:** C. They protect cookies from direct access by client-side scripts, mitigating some XSS-based theft attempts.
4. **Answer:** B. CSRF exploits the fact that browsers automatically attach cookies to outgoing requests for a domain.
5. **Answer:** B. Stripping or sanitizing potentially dangerous HTML and script tags from user-generated content.
6. **Answer:** C. A malicious link contains script code that the target server includes in its error message, immediately returning it to the user's browser.
7. **Answer:** C. To verify that an HTTP request originated from the legitimate site's page/form rather than a third-party injection.
8. **Answer:** B. Using parameterized (prepared) statements that keep SQL logic separate from user input.
9. **Answer:** B. Ensure a cookie is only sent over HTTPS connections.

10. **Answer:** B. Sanitizing or encoding any untrusted data before adding it into the DOM or URL fragments.
-

Part B

1. HTTP Cookies and Authentication

- **Why critical?** Cookies provide continuity across stateless HTTP by storing session identifiers or login states.
- **Attacks:**
 1. **Cookie theft** via XSS or malicious script reading `document.cookie`.
 2. **Network sniffing** if no HTTPS is used.
 3. **Session fixation** or replay if cookies aren't invalidated or bound to session constraints.
- **Defenses:**
 1. **TLS** (HTTPS) to prevent interception.
 2. **Cookie binding** with IP or user agent checks, or cryptographic signing of cookie content.
 3. **Short session lifetimes** plus re-auth on critical actions.

2. Mixed Content and HTTPS

- Mixed content is dangerous because an attacker controlling or hijacking the HTTP elements can inject malicious scripts or modify content.
- If the attacker intercepts an HTTP script, they can rewrite it to perform malicious actions, effectively defeating the security of the main HTTPS page.
- This breaks the authenticity and integrity guarantees of HTTPS.

3. CSRF vs. XSS

- **CSRF:** Exploits automatic cookie sending to cause unintended actions on a site where the user is already authenticated.
- **XSS:** Involves injecting scripts into a trusted site's pages so they run in a victim's browser.
- A site can be vulnerable to both: XSS can steal user cookies or embed hidden forms (CSRF).

- Mitigations:
 - **CSRF**: Use secret tokens, SameSite cookies, user re-auth.
 - **XSS**: Input/output sanitization, Content Security Policy, HttpOnly cookies.

4. Subdomain Scoping of Cookies

- **Domain** attribute = `.example.com` → the cookie is accessible to all subdomains under `example.com`.
- **Path** attribute further restricts which URL paths can see the cookie.
- **Vulnerability**: If you set `.example.com` but there's an untrusted subdomain (`untrusted.example.com`), that subdomain can read the cookie, leading to a compromise of user sessions.

5. SQL Injection Attack Patterns

- **Advanced Patterns**:
 1. **UNION-based injection**: `UNION SELECT password,1,1 FROM admin_table` appended to the query.
 2. **Time-based blind injection**: Use `SLEEP(5)` or `WAITFOR DELAY '0:0:5'` to test values.
- Naive sanitization might fail because attackers can nest or obfuscate queries (hex-encode, comments, etc.).

6. Prepared Statements vs. Escaping

- Prepared statements separate the SQL logic from the user input. The DB engine handles placeholders safely.
- **Example** (pseudo-code):

```
stmt = connection.prepare("SELECT * FROM users WHERE
username=? AND password=?")
stmt.bind(usernameInput)
stmt.bind(passwordInput)
result = stmt.execute()
```

This approach ensures user input can't break the query structure.

7. CSP (Content Security Policy)

- CSP can specify `default-src 'self'` to restrict resources to the same origin.
- `script-src 'self' 'nonce-xyz'` can limit scripts to those with a particular nonce or from a trusted domain.
- This drastically reduces or blocks inline scripts or external resources from unknown domains, mitigating many XSS attacks.

8. Impact of XSS on SOP

- If an attacker injects malicious JavaScript into the legitimate domain, that code runs **with** that domain's origin privileges.
- They can read or modify any data accessible under that origin: cookies, local storage, or even the site's private user data.
- Essentially, it's as if the site's own script performed those actions, bypassing the SOP.

9. Browser Security vs. Server Security

- **Client-side:** SOP, XSS filters, sandboxing, same-site cookies, etc.
- **Server-side:** Input validation, encryption, session management.
- Relying on only one side is insufficient because malicious actors can bypass client checks or exploit server misconfigurations. Multiple layers (defense in depth) ensure if one layer fails, others still protect the system.

10. Designing a Secure File-Upload

- **Measures:**
- Validate file type and MIME type (e.g., only allow image/jpeg).
- Rename the file on the server (avoid user-supplied filenames).
- Store files outside the webroot or serve them via a separate domain to prevent direct script execution.
- Perform virus scans or sanitize images (strip EXIF or metadata).
- Set correct **Content-Type** headers on download to avoid forced execution as HTML.

Part C

1. Estimating Cookie Expiration Risks

- (a) Probability of success in 4 hours:
 - Attacker success rate $\sim (1/2^{\{,\}000\{,\}000})$ per hour.
 - Over 4 hours, an approximate bounding approach:

$$\begin{aligned}
P(\text{success in 4 hrs}) &\approx 1 - (1 - p)^4 \text{ with } p = 1/2,000,000. \\
&1 - (1 - 0.5 \times 10^{-6})^4 \\
&\approx 1 - (1 - 0.5 \times 10^{-6})^4 \\
&\approx 1 - (0.9999995)^4 \\
&\approx 2 \times 10^{-6}
\end{aligned}$$

(this is a rough estimate, still around 1 in 500,000).

- (b) With Max-Age = 1 hour, the probability is roughly:
 $1 - (1 - 0.5 \times 10^{-6})^1 \approx 0.5 \times 10^{-6}$.

Reducing session length cuts success probability proportionally in this scenario.

2. SQL Injection and Database Size

- (a) **Worst case:** enumerating 1,000,000 rows × each row's username of max length 10, from 95 characters. For each character, attacker does a 1-second test (timing-based): $1,000,000 \text{ rows} \times 10 \text{ chars/row} \times \log_2(95) \text{ tests?}$

Actually, a naive approach might do a binary search per character $\log_2(95)$ 6.57. Then total checks:

$$\begin{aligned}
1,000,000 \times 10 \times 7 \times 1 \text{ s} &\approx 70,000,000 \text{ s} \\
&\approx 2.22 \text{ years.}
\end{aligned}$$

(This is a simplified estimate. If they do single attempts per possible char, it might be even more.)

- (b) **Reduction:** Use faster out-of-band channels, exploit advanced queries (dump many rows at once), or find a direct injection that enumerates entire tables with fewer queries (e.g., union-based injection or more efficient conditional checks).

3. XSS Injection Surface

- (a) Each payload ~100 chars; total limit = 2,000 chars.
 $\frac{2000}{100} = 20$ possible attempts embedded.
- (b) Even if < and > are escaped, attackers might use other encodings (e.g., < for <) or manipulate attributes (e.g., onerror in). They could also exploit unescaped quotes or use JavaScript protocol URIs if not filtered.

4. Reflected XSS via URL Parameters

- Each request test: ~500ms to see result. In 1 minute =
60 seconds →
 $\frac{60 \text{ s}}{0.5 \text{ s/request}} = 120 \text{ attempts}$
if done sequentially.
